HWDC 2026

同一個 Prompt 為什麼差這麼多？——從踩坑到 Harness Engineering

Olivia Shiu

Solutions Architect

Amazon Web Services

SPEAKER · 先認識一下

Olivia Shiu

Solutions Architect · Amazon Web Services

THEN

工程師

從 UEFI 韌體到 K8s、AWS 雲端服務、專案經理

NOW

架構師

幫企業設計架構、發想策略，也是聆聽者

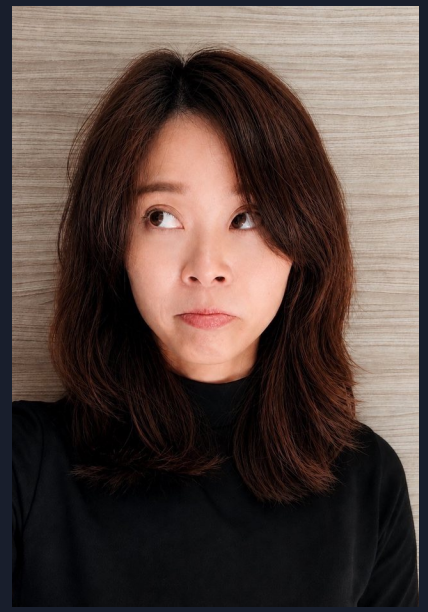


© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Agenda

- Prompt Engineering
  —— 學會「怎麼問」AI
- Context Engineering
  —— 給 AI 對的資訊，且在對的時間點
- Harness Engineering
  ——當 AI 開始「自己動」需要控制 AI 的方向

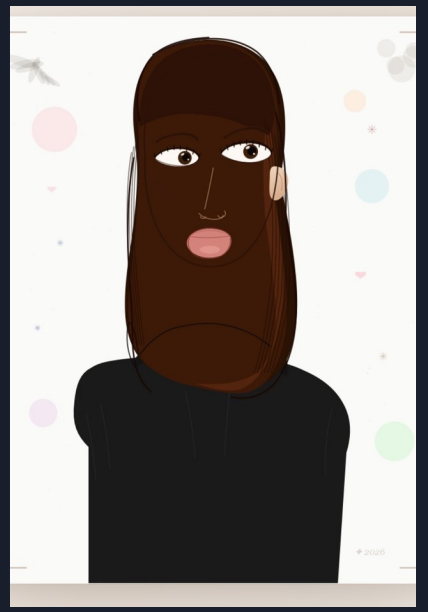
同一個 Prompt，為什麼差這麼多？--馬鈴薯事件



原圖



ChatGPT · GPT-4o



Claude 3.7 Sonnet · SVG

圖片為 2025.05.05 講者實際試用結果，Claude 3.7 Sonnet 當時無原生生圖能力

CHAPTER · 01

Prompt Engineering

學會「怎麼問」——AI Engineering 的第一步

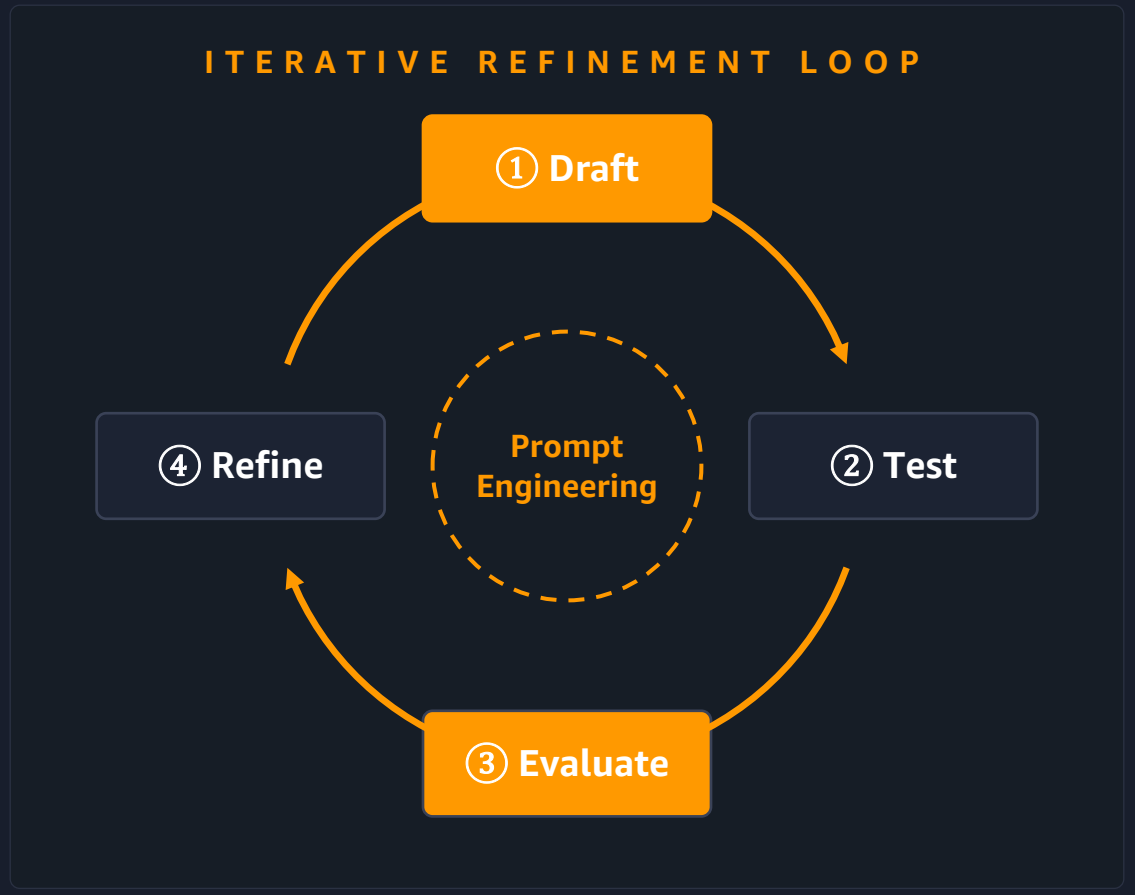
DEFINITION · 2 OF 2

什麼是 Prompt Engineering？

> *“Prompt engineering 是引導生成式 AI 產出預期結果的過程”*
>
> *“Prompt engineering is the process where you guide generative AI solutions to generate desired outputs.”*
>
> — aws.amazon.com/what-is/prompt-engineering

透過**設計與優化 prompt**，引導 AI 產出更精準、更符合預期的結果

這是一種可被學習、可被工程化的過程



好 Prompt vs 壞 Prompt



清晰、有結構

可重現、可驗證

輸入、處理邏輯、輸出格式都寫清楚，產出就會**穩定可重現**；GenAI 被引導去做**結構化推理**，整條鏈路都**可控、可驗證**。



模糊、無結構

AI 自由發揮

沒講清楚要什麼時，AI 會自己**補細節**——結果常常是**幻覺**、每次**不一致**，無法重現也**無法驗證**。

好 Prompt 的三大要素

缺一不可——這是把「問問題」變成「工程」的最小可行結構。



Input

給 AI 看什麼

備齊要處理的資料與告訴 AI **背景資料**、指定它扮演什麼**角色**、劃出可做不可做的**界線**，再附上幾個**範例**讓它照著走。



Process

要 AI 怎麼想

把思考過程拆成可驗證的**步驟**，訂下每步該遵守的**規則**，遇到分岔時給出明確的**決策依據**，不要讓它自己猜。



Output

讓 AI 交什麼

指定要交出的**格式**（表格、JSON、Markdown），列出必要**欄位**、設定**長度**上限，並說明期望的產出與**風格**。

DEMO · 1a STABILITY TEST

旅平險比較 — 跑三次看穩定度

✕ Unclear PROMPT

User

幫我比較一下旅平險。

Run 1	Run 2	Run 3
列出 3 家	列出 5 家	離題

產出發散、欄位不一、難以比較。

✓ STRUCTURED PROMPT

User

[Input] 比較 A/B/C 三家旅平險，10 天歐洲行。

[Process] 依保額、自負額、除外條款逐條比對。

[Output] Markdown 表格 + 推薦理由 ≤ 100 字。

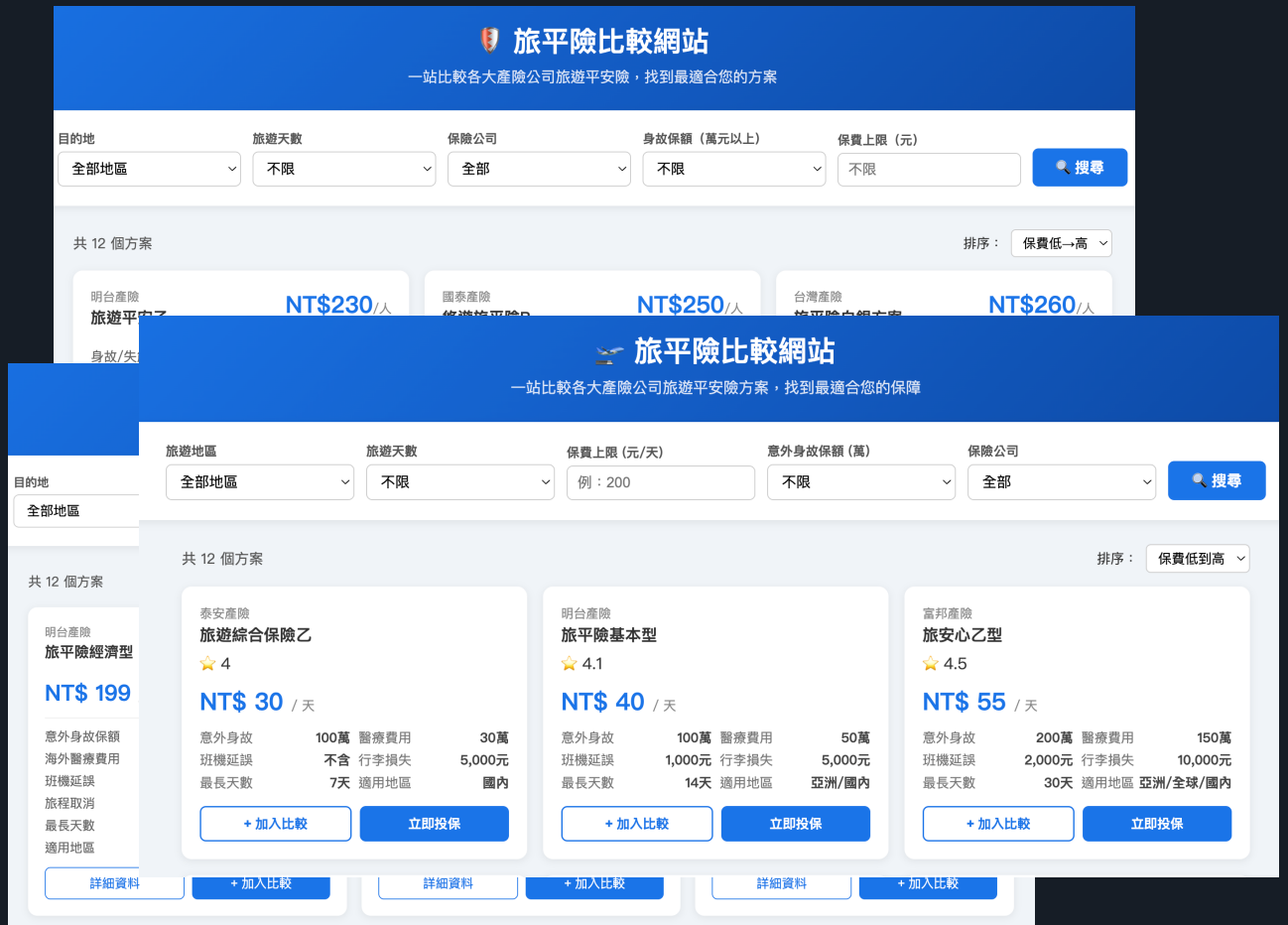
Run 1	Run 2	Run 3
表格 + 結論	表格 + 結論	表格 + 結論

三次結果一致，可重現、可驗證。

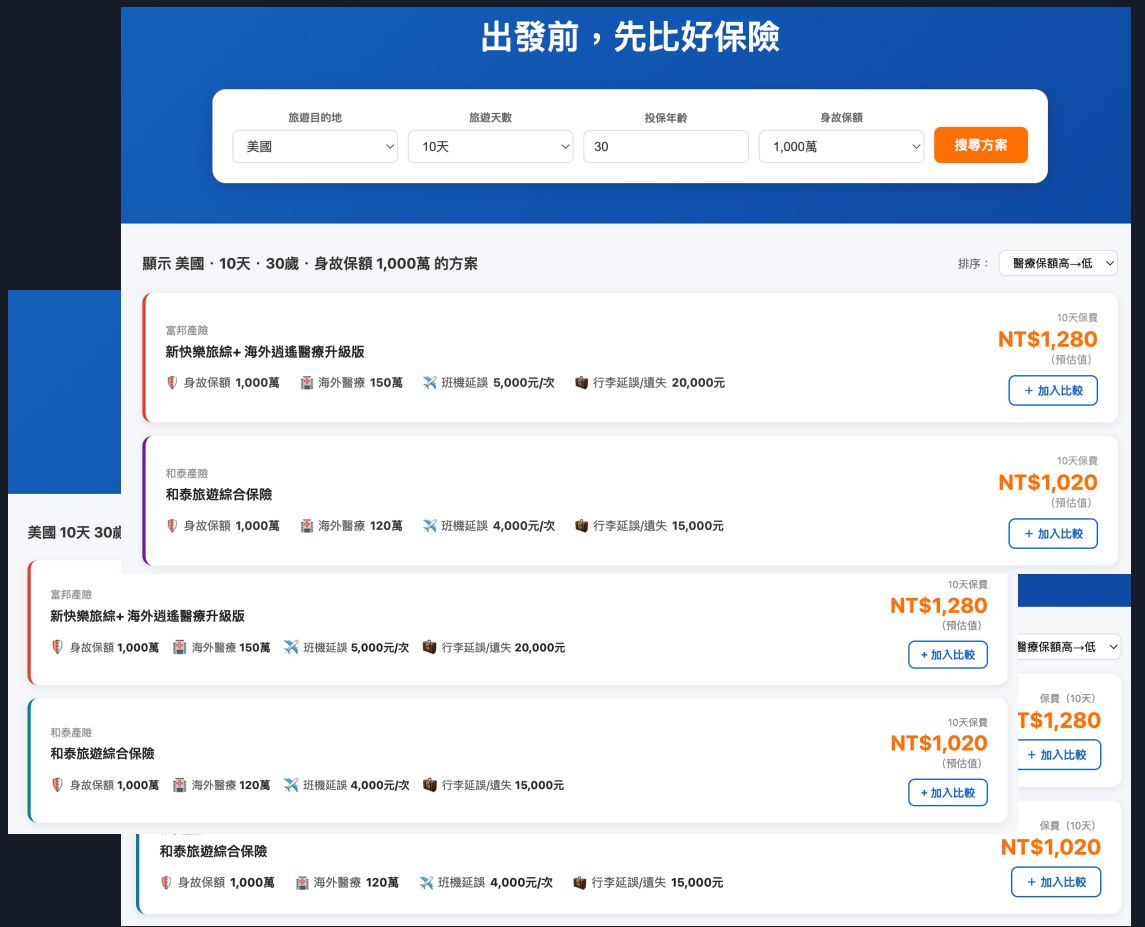
DEMO · 1a STABILITY TEST

旅平險比較 — 跑三次看穩定度

✕ Unclear PROMPT



✓ STRUCTURED PROMPT



DEMO · 1b HALLUCINATION TEST

Legacy Code 分析 — 簡單 prompt 容易發散

✕ ONE-LINER

User

幫我看一下這段 COBOL。

Assistant

// 推測為 1990s 銀行系統...

// 假設使用 DB2（未驗證）

// 提到不存在的 paragraph：CALC-VAT

→ AI **編造細節、發散**，無法信任。

✓ STRUCTURED PROMPT

User

[Input] COBOL 檔案 + JCL job 設定

[Process] 1) 列 paragraph 2) 標 I/O 3) 找未用變數

[Output] JSON：{paragraphs, io, dead_vars}

Assistant

```
{ "paragraphs": ["MAIN-RTN", "READ-FILE"],
  "io": [{"file":"CUST.DAT","mode":"IN"}],
  "dead_vars": ["WS-TEMP-2"] }
```

→ **結構化、可驗證、可被下游程式消費。**

DEMO · 1b HALLUCINATION TEST

Legacy Code 分析 — 簡單 prompt 容易發散

✕ ONE-LINER

檔案	大小	Transaction ID	功能定位
`COSGN00C.cbl`	10 KB	CC00	登入畫面
`COMEN01C.cbl`	12 KB	CM(	
`COACTVWC.cbl`	75 KB	CAV	

檔案	
`COCOM01Y.cpy`	CICS COMMA
`COMEN02Y.cpy`	主選單 11 項選
`COTTL01Y.cpy`	畫面標題共用
`CSMSG01Y.cpy` / `CSMSG02Y.cpy`	系統訊息常數
`CSUSR01Y.cpy`	使用者安全資
`CVCUS01Y.cpy`	客戶主檔
`CVACT01Y.cpy`	帳戶主檔
`CVACT02Y.cpy`	信用卡主檔 `CARD-RECORD` (150 bytes)
`CVACT03Y.cpy`	卡片交叉參照 `CARD-XREF-RECORD` (50 bytes)
`CVCRD01Y.cpy`	卡片作業共用工作區 `CC-WORK-AREAS`

3.1 主要實體與欄位

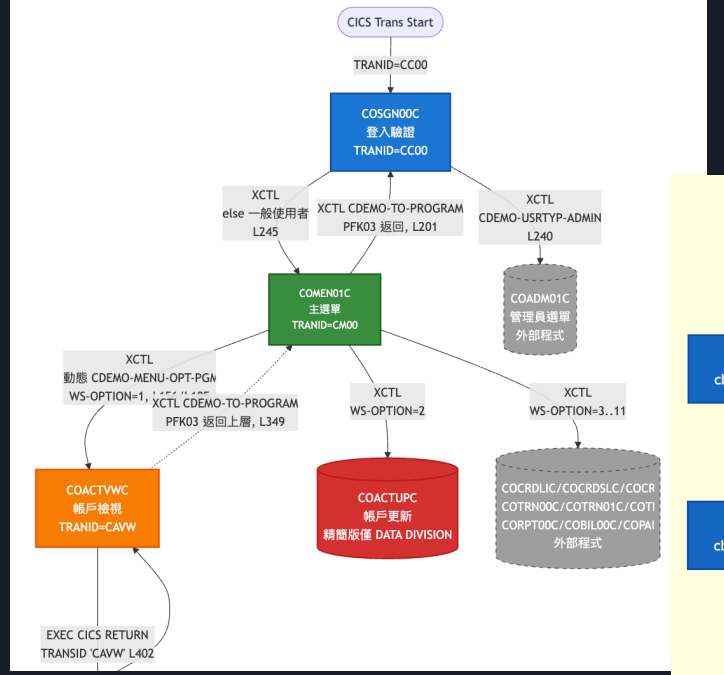
```
SEC-USER-DATA (CSUSR01Y, 80 bytes)
  SEC-USR-ID          X(08)  ← 主鍵
  SEC-USR-FNAME       X(20)
  SEC-USR-LNAME       X(20)
  SEC-USR-PWD         X(08)  ← 密碼 (明文)
  SEC-USR-TYPE        X(01)  ← 'A'=Admin, 'U'=User

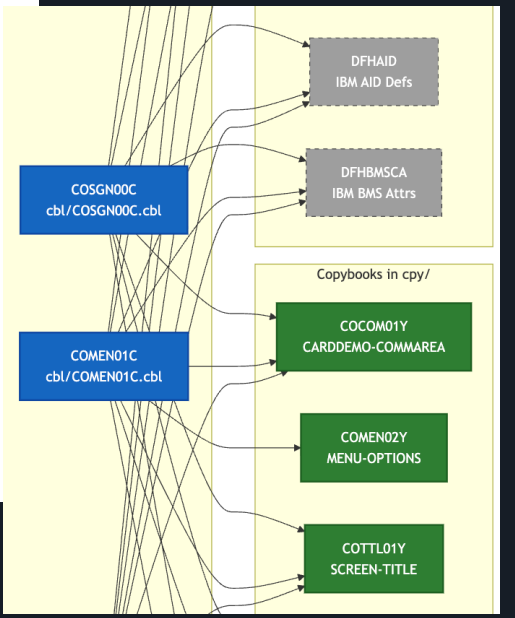
CUSTOMER-RECORD (CVCUS01Y, 500 bytes)
  CUST-ID                     9(09)  ← 主鍵
  CUST-FIRST/MIDDLE/LAST-NAME  X(25) ×3
  CUST-ADDR-LINE-1/2/3        X(50) ×3
  CUST-ADDR-STATE-CD          X(02)
  CUST-ADDR-COUNTRY-CD        X(03)
  CUST-ADDR-ZIP               X(10)
```

✓ STRUCTURED PROMPT

2. 系統架構圖（Mermaid）

依據實際的 `EXEC CICS XCTL` 、`EXEC CICS RETURN TRANSID(...)` 與 BMS map 引用繪製。虛線表示透過 PFK03（返回上





THE CEILING

Prompt Engineering 的天花板

寫得再好，也有 prompt 解不了的情況。

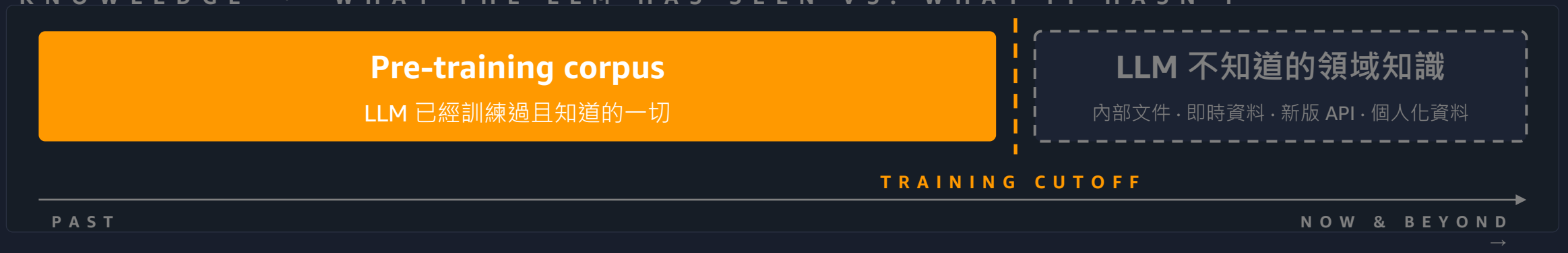
- **公司內部文件**：LLM 沒看過
- **即時資訊**：今天的股價、最新匯率
- **新版 API**：訓練截止後才發布
- **個人化資料**：行事曆、訂單

ROOT CAUSE

問題不在 prompt 寫得好不好——

而是 LLM 根本沒有這些知識。

KNOWLEDGE · WHAT THE LLM HAS SEEN VS. WHAT IT HASN'T



WHAT'S NEXT

把正確的知識，在正確的時間，交給 LLM。

COMING UP IN CHAPTER 2 —— Context Engineering →

CHAPTER · 02

Context Engineering

給 AI 對的資訊——且在對的時間點

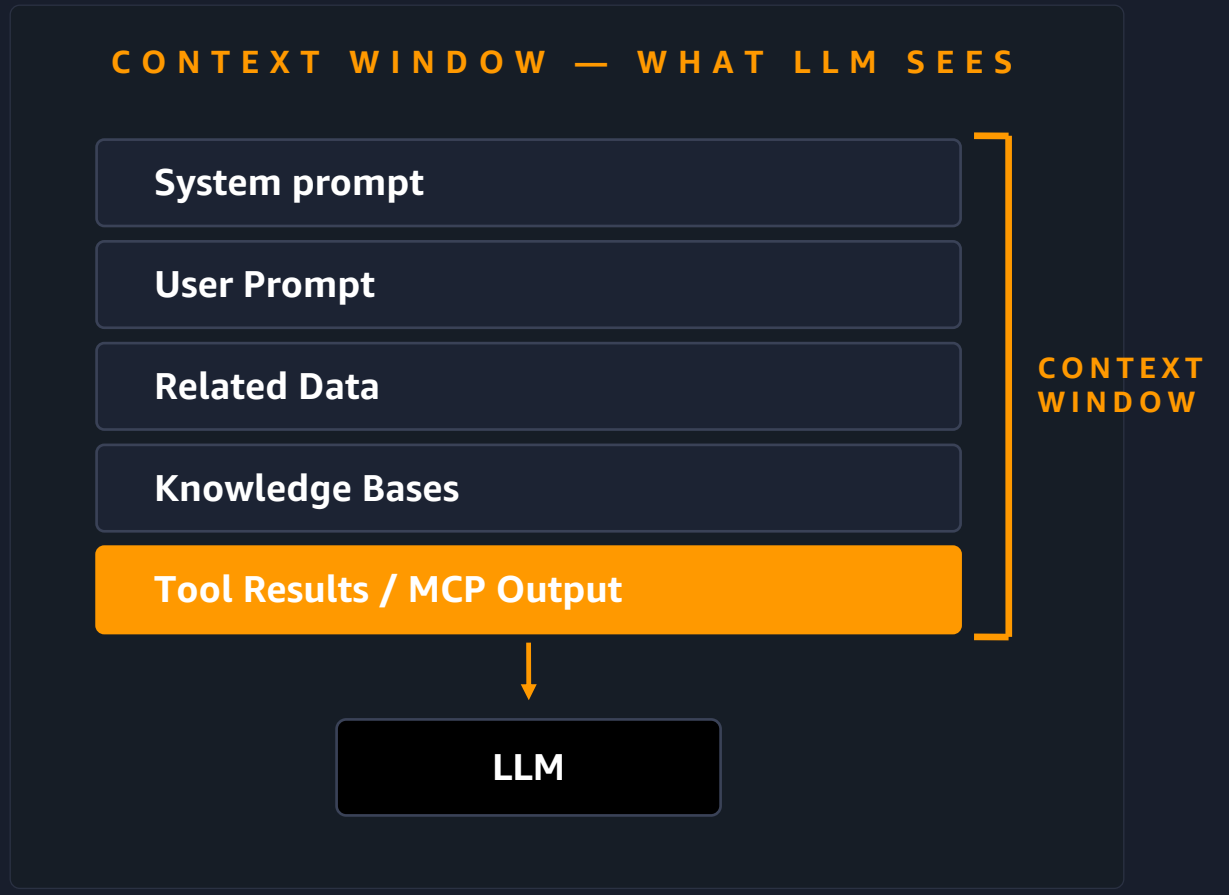
DEFINITION · 1 OF 2

什麼是 Context？

> *“Context 指的是 LLM 在生成回應時，實際納入計算的所有 token ——也就是上下文內容”*
>
> *“Context refers to the set of tokens included when sampling from a large-language model.”*
>
> — anthropic.com/engineering/effective-context-engineering

也就是 LLM 在回答時，**「實際看到」的所有資訊**

不只是 prompt——包括系統指令、檢索結果、工具回傳、歷史訊息等等



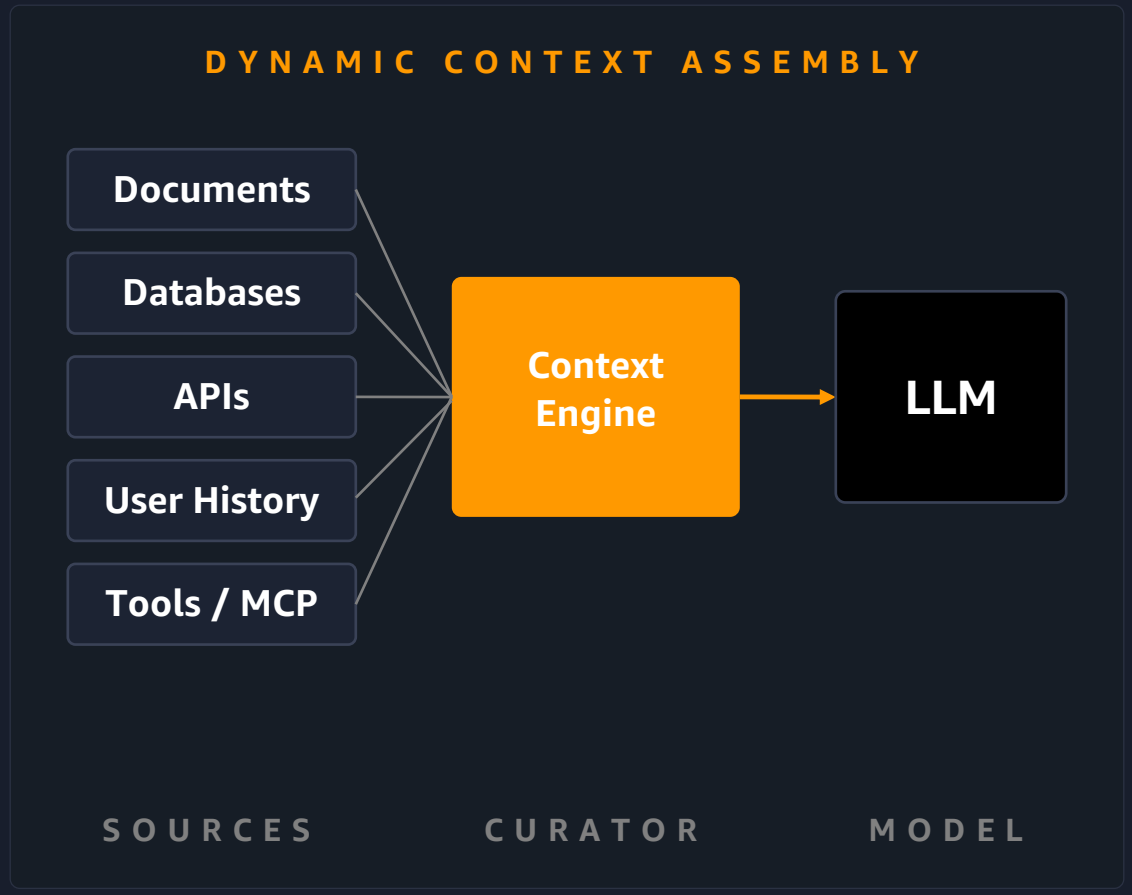
DEFINITION · 2 OF 2

什麼是 Context Engineering？

> *“為 LLM 動態組裝執行任務所需最佳資訊的系統化實踐”*
>
> *“The practice of designing systems that dynamically assemble the optimal set of information for an LLM to perform a given task.”*
>
> — anthropic.com/engineering/effective-context-engineering

不是把所有資料丟給 AI——而是**動態組裝最適合當前任務的資訊**。

把它想成 AI 的「工作記憶體管理」。



THE COST OF DOING IT WRONG

為什麼要做 Context Engineering？

↑

Token 爆炸

Context window 一旦塞滿，LLM 容易「忽略中間段」（lost-in-the-middle）

$$$

成本上升

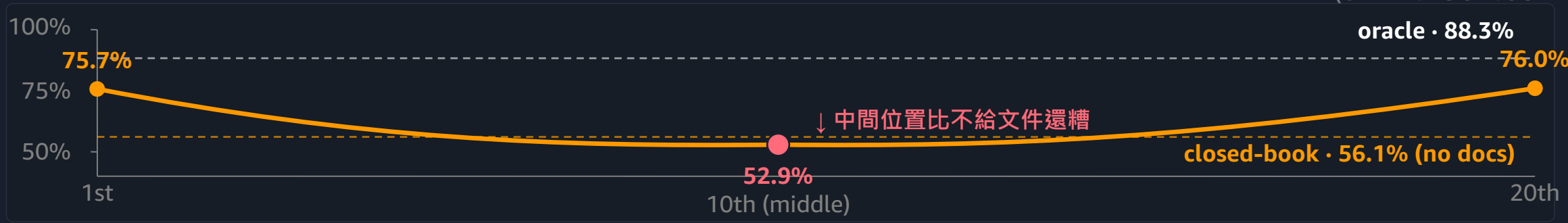
Token 越多，費用越高，回應越慢。每次請求都在燒錢

↓

準確度下降

無關資訊干擾推理，反而稀釋了真正重要的訊號

Multi-doc QA · Liu et al. 2023 (arXiv:2307.03172)



把 20 份文件全塞給 LLM，答案在中間時準確度 52.9% —— 比完全不給文件還差（56.1%）。證明 context 不是越多越好，位置和篩選才是關鍵。

即使是 1M context 的最新模型，**lost-in-the-middle 依然存在**——多篇研究指出這是 transformer attention 的**結構性問題**，不是靠訓練量或更大 context 就能消除。

(arXiv:2502.01951 · 2508.07479 · 2510.10276)

THE NAIVE APPROACH

最直覺的做法——直接餵文件

要讓 LLM 擁有 domain knowledge？
把整份文件貼進 prompt，複製貼上就能 demo，不用建 infra、不用寫 RAG

直覺、有效、最快可以驗證——也是大多數 PoC 的起點

BUT… 第 2 份、第 10 份、第 100 份文件呢？
Demo 能跑，但不具可擴展性，難以應用在Production

prompt.txt · HR-assistant

```
# System
You are an HR policy assistant.
Answer strictly from the handbook below.

# Handbook (pasted)
"""
第一章　總則……
第二章　工作時間……
第三章　請假辦法……
…（整份 200 頁員工手冊貼在這裡）…
第八章　離職與交接……
"""

# Question
育嬰假怎麼申請？
```

DEMO · 2a DOCUMENT AS CONTEXT

把文件當 Context 餵給 LLM

USER PROMPT

System
你是 HR policy assistant，根據 HR 手冊回答問題。

我是到職 1 年 3 個月的員工，明年計劃：3 月結婚（請幫我規劃蜜月旅行的休假）、6 月太太預產期、8
月想帶父母出國旅遊 10 天、11 月父親預計住院手術需要陪病。請幫我規劃整年度的休假計畫，告訴我每項假別可以請幾天、薪資怎麼算、需要什麼文件、多久前要申請。最後算出我總共會被扣多少薪水（如果有）。

Handbook (40k tokens)
「員工手冊 2026 版」全文…

RESULT

ASSISTANT

✓根據員工手冊，以下是你明年（2027年）的完整休假規劃：

月份	事由	假別	天數	薪資
3月	結婚	婚假	8天	全薪
3月	蜜月旅行	特休	2天	全薪
6月	太太生產	陪產假	7天	全薪
8月	帶父母出遊	特休	6天	全薪
11月	父親陪病	家庭照顧假	7天	**無薪**
11月	父親陪病	特休（剩餘）	2天	全薪
合計			**32天**	

THE PROBLEM WITH NAIVE APPROACH

直覺，但無法 scale。



手動操作

每次都要找對檔案、整份貼進來。



無法擴展

100 份文件、跨系統資料怎麼辦？



沒有時效性

文件版本一更新就過期。



context 沒被設計

不是組裝出來的，是被「堆」上去。

→ 我們需要更系統化的方式來管理 context。

THE NORTH STAR

在對的時間，給對的資訊，到對的模型。

這就是 Context Engineering 的核心——同時優化準確度與成本。

系統化管理 Context 的兩個概念



MCP

拿到什麼資料

Model Context Protocol 是一套開放**標準協定**——以統一介面的方式連接資料庫、SaaS API、檔案系統，並且可**跨 LLM 重複使用**



Skill

怎麼用這些資料

Skill 是寫給 AI 看的**操作手冊**——將「觸發條件、執行步驟、工具呼叫、輸出格式」封裝成**可複用模板**，由 AI 依情境動態載入

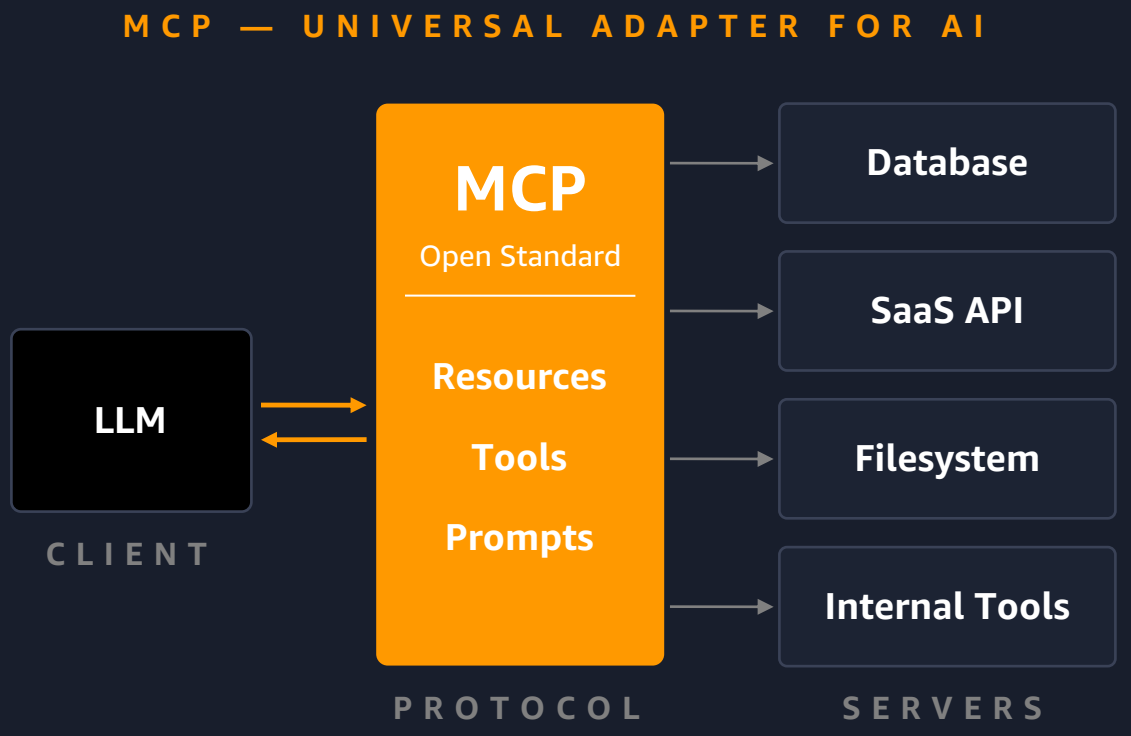
MECHANISM · 01 · MCP

什麼是 MCP？

> *“MCP 是一套開放標準，讓 AI 以統一語言與外部資料源、工具與服務溝通”*
>
> *“MCP is an open standard that creates a universal language for AI to communicate with external data sources, tools, and services.”*
>
> — aws.amazon.com/blogs/machine-learning

不用重複為每個資料源客製化寫 Connector

一次寫好 MCP server，所有 LLM 都能用



MECHANISM · 02 · SKILL

什麼是 Skill？

> *“Agent Skills 是一種輕量、開放的格式，以專業知識與工作流程擴展 AI Agent 的能力”*
>
> *“Agent Skills are a lightweight, open format for extending AI agent capabilities with specialized knowledge and workflows.”*
>
> — agentskills.io

把「如何完成一項任務」封裝為可動態載入的程序性知識模組，讓 GenAI 能依情境隨需呼叫

skills/check_calendar/SKILL.md

```
---
name: check_calendar
when: 使用者問行程相關問題
---

# Steps
1. 取得使用者時區
2. 呼叫 calendar.list(today)
3. 回傳：時間 + 標題 + 地點
```

MCP VS SKILL

不是二選一——兩者互補

	MCP	SKILL
核心職責	拿到什麼資料	怎麼用這些資料完成任務
技術定義	連接外部服務的 **標準協定**	封裝任務流程的 **知識手冊**
適用情境	需要外部資料 / 工具	需要重複執行特定流程
載入時機	啟動 CLI 時連線常駐可用	依情境動態載入

IF WE WANT MORE AUTOMATION...

把 Context、MCP、Skill ——
全部裝進一個 Agent。

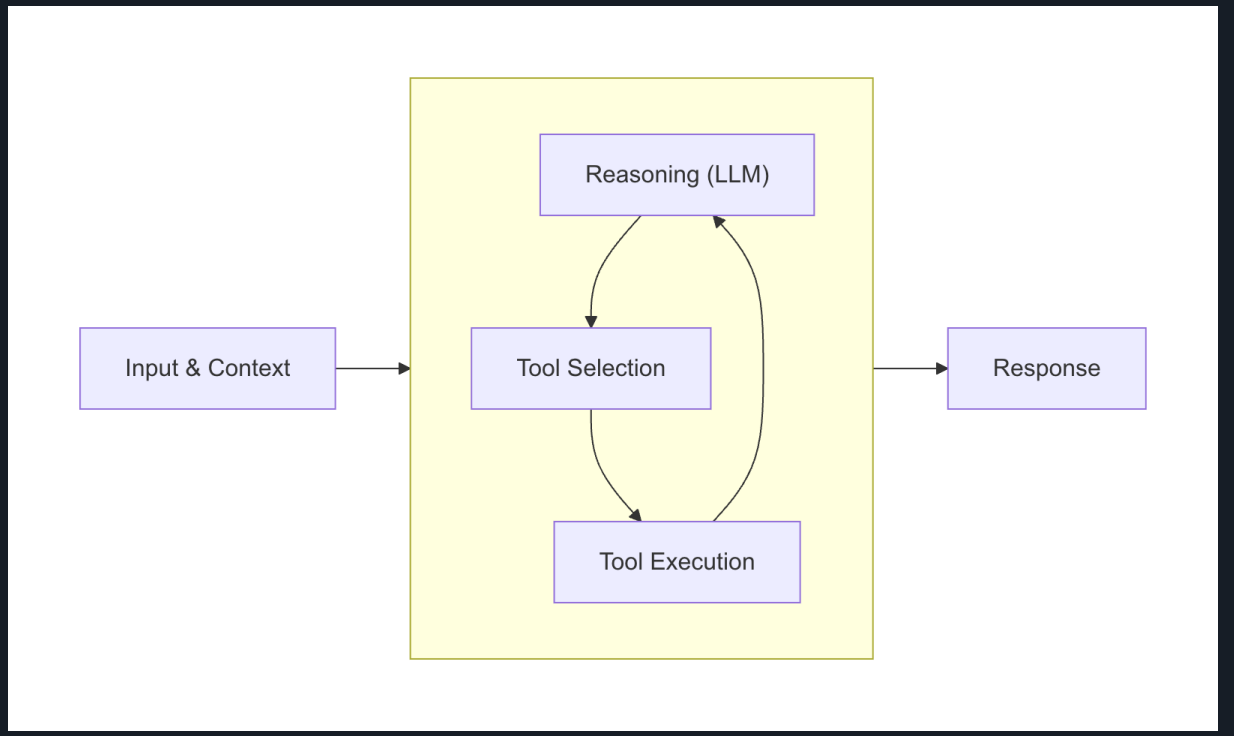
讓它自主規劃、執行、完成複雜任務。

DEFINITION

什麼是 Agent？

> *“Agent 是一種軟體程式，能與環境互動、蒐集資料，並運用這些資料自主執行任務，以達成預定目標。“*
>
> *“An agent is a software program that can interact with its environment, collect data, and use that data to perform self-directed tasks that meet predetermined goals.”*
>
> — aws.amazon.com/what-is/ai-agents

AGENT LOOP



單一 Agent 無法包辦所有任務



範圍清楚、單一專業

SINGLE AGENT WORKS WHEN...

當任務是**單一資料源**的查詢、或**輸入輸出格式明確**的處理，一個 agent 就能穩定交付，不需要協作。



跨專業、長流程任務

SINGLE AGENT BREAKS WHEN...

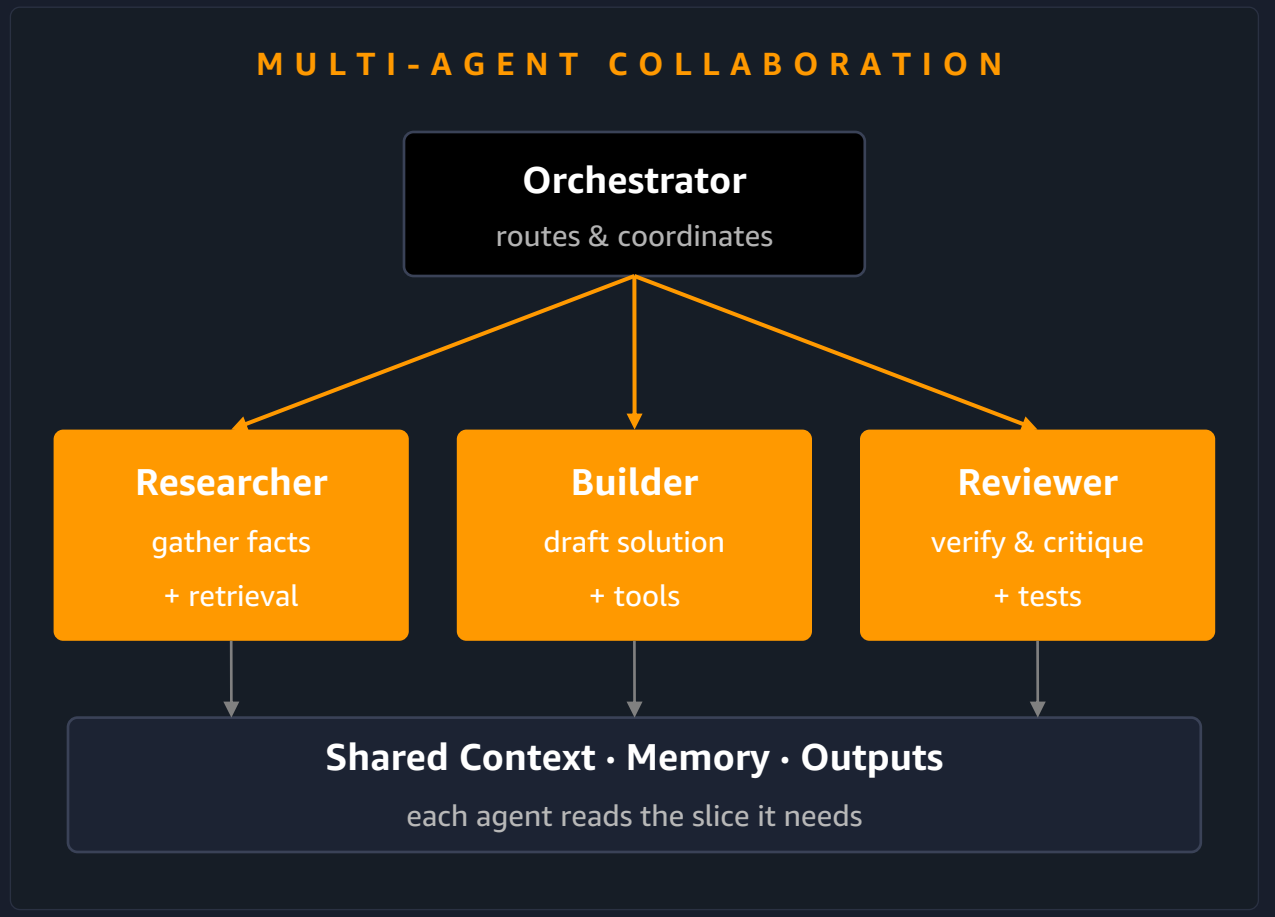
要同時**寫程式、測試、部署**，或需要**多種協作任務**時，單一 agent 的 context 會滿、產出會發散、品質難以收斂。

MULTI-AGENT COLLABORATION

什麼是 Multi-Agent？

> *“Multi-Agent 是一種協作模式——多個自主 Agent 各具不同的角色、專長或目標，透過分工解決複雜任務”*
>
> *“A pattern in which multiple autonomous agents, each with a distinct role, specialization, or objective, negotiate to solve complex tasks.”*
>
> — aws.amazon.com/prescriptive-guidance

把一個大任務拆成小任務，並交由**多個專業 Agent 分工協作**。



THE CEILING

Context Engineering 的天花板

Agent 動起來了。但你會發現——

- **同樣 context，時好時壞**
- 手動執行成功，**自動化卻不穩定**
- 出錯時，**不知道哪一步壞了**

ROOT CAUSE

我們只關注了「給 AI 什麼」——

卻沒有設計**「如何控制 AI」**。

SAME INPUT, THREE RUNS · WHY CONTEXT ALONE ISN'T ENOUGH

RUN 1 ✓ SUCCESS	RUN 2 △ PARTIAL	RUN 3 ✗ FAIL
✓ plan	✓ plan	✓ plan
✓ retrieve docs	✓ retrieve docs	✗ wrong doc retrieved
✓ call tool	△ tool timeout	✗ hallucinated answer
✓ final answer	△ answer incomplete	? no trace, can't debug

同樣的 input、同樣的 context——結果卻每次不同。這是 context engineering 無法解決的問題。

WHAT'S NEXT

從「給對的資訊」，到「建好控制系統」。

COMING UP IN CHAPTER 3 —— Harness Engineering →

CHAPTER · 03

Harness Engineering

當 AI 開始「自己動」——你需要韁繩、儀表板、與流程，才能控制 AI 的方向

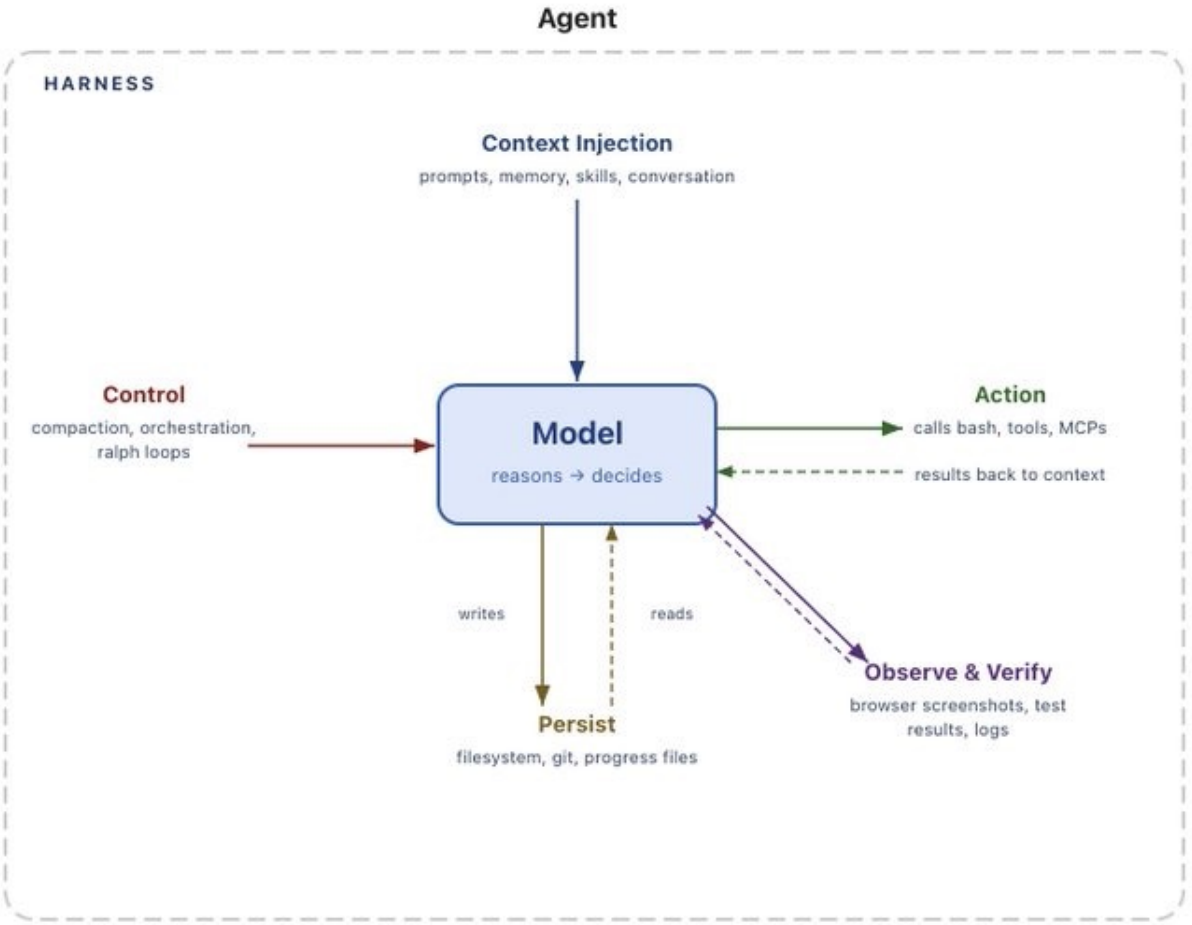
DEFINITION · 1 OF 2

什麼是 Harness Engineering？

> *“Harness 如今已成為業界通稱，泛指 AI Agent 中除模型以外的所有組件——一句話概括：*
>
> *Agent = Model + Harness”*
>
> *“The term harness has emerged as a shorthand to mean everything in an AI agent except the model itself - Agent = Model + Harness.”*
>
> — martinfowler.com/articles/harness-engineering.html

Prompt 教 AI **怎麼說**。Context 給 AI **看什麼**

Harness 決定 AI **能做什麼、做錯了怎麼辦**



https://www.langchain.com/blog/the-anatomy-of-an-agent-harness

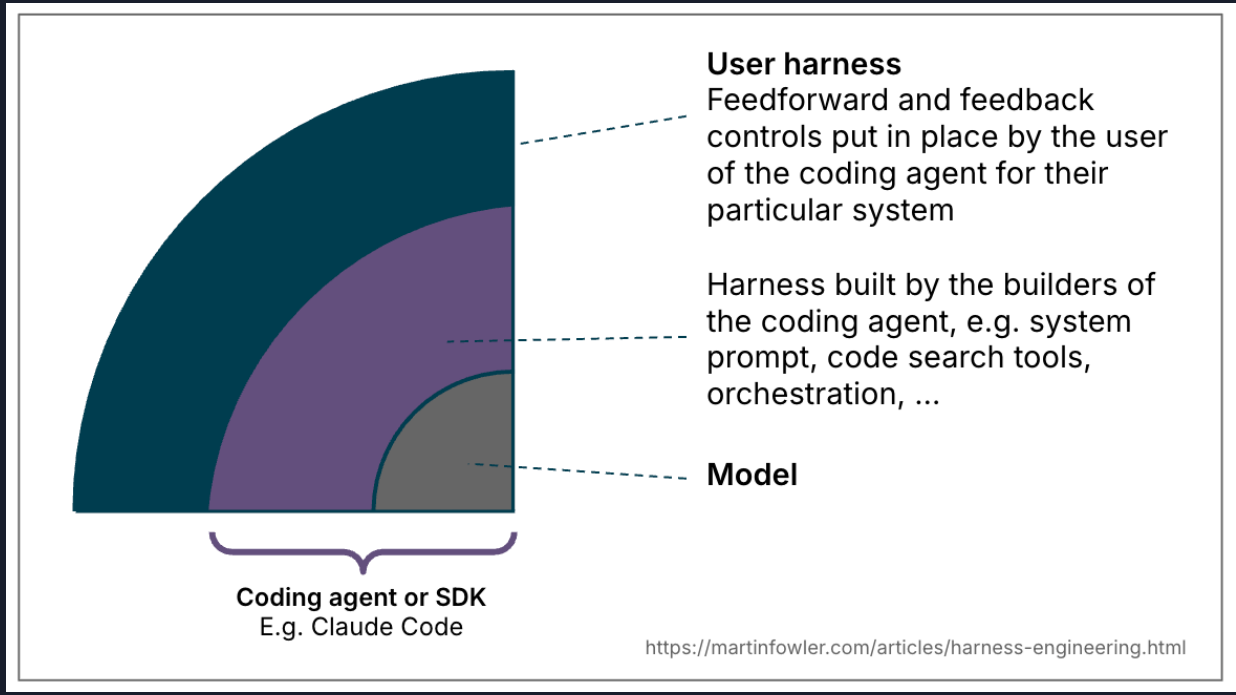
DEFINITION · 2 OF 2

為什麼叫 “Harness”？

Harness (馬具)，用來駕馭強大、但**無法完全預測**的動物

如同 LLM 能力強大，但需要一套**控制系統**才能穩定駕馭方向

LLM 本質上具備**不確定性**，因此在 Production 環境時，我們需要**圍繞它來建立控制系統**



THE THREE LAYERS

完整的 AI Engineering 心智模型

由下而上構築

LAYER 3
Harness
建構控制系統，讓 AI **能上線運行**
Orchestration · Eval · Observability · Recovery

LAYER 2
Context
動態組裝資訊，讓 AI **擁有正確知識**
RAG · MCP · Skills · Agents · Multi-agent

LAYER 1
Prompt
設計明確的指令，讓 AI **聽得懂**
Structured I/O · Logic · Process

Harness 的六大核心

對應 Anthropic 的 **Planner** · **Generator** · **Evaluator** 三階段 loop。



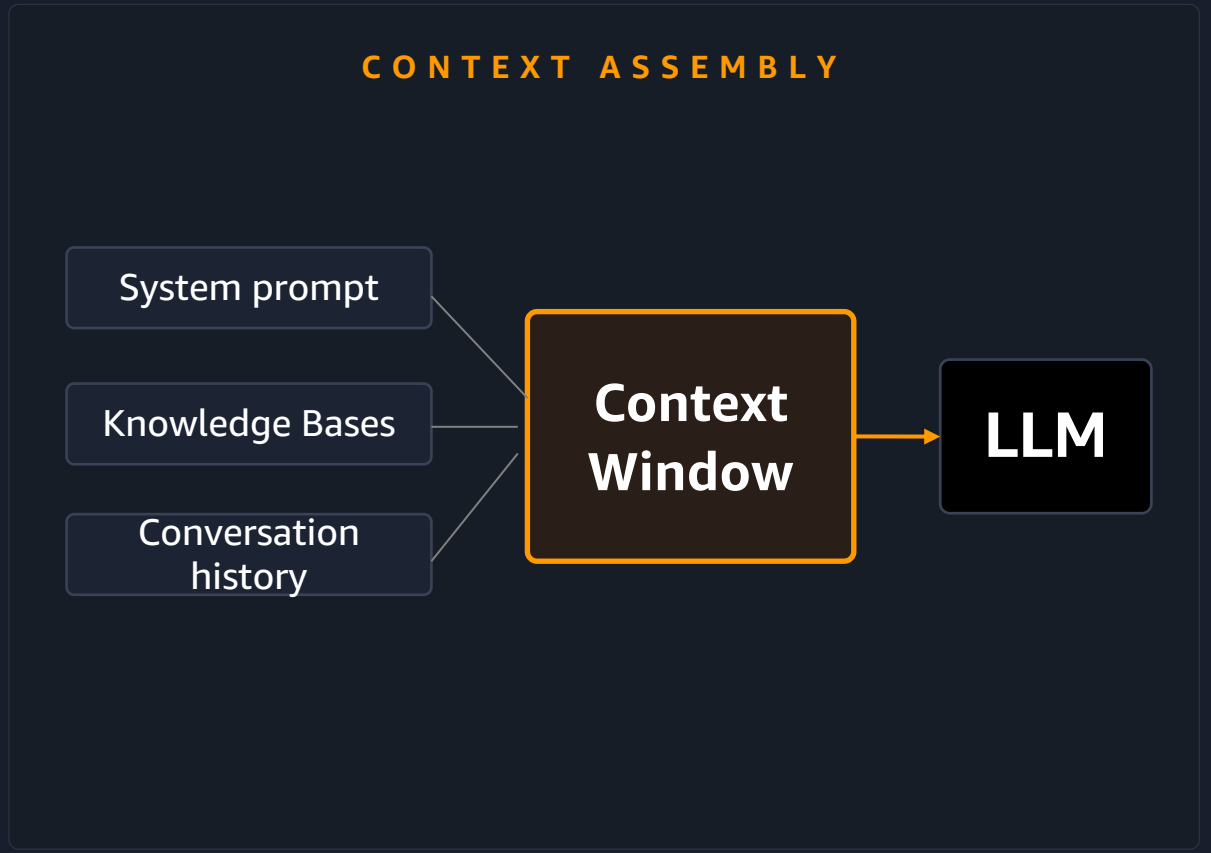
PILLAR 01 · PROMPT & CONTEXT

上下文工程——給 AI 看對的資訊

Harness 的**第一層核心**——AI 看到什麼，決定它能做什麼。

- **System Prompt**：定義角色、邊界、I/O
- **Knowledge Bases**：給予正確的文件與資料
- **Conversation History**：對話歷史資料

提供 LLM 正確的資訊，並讓它在可控的邊界內思考



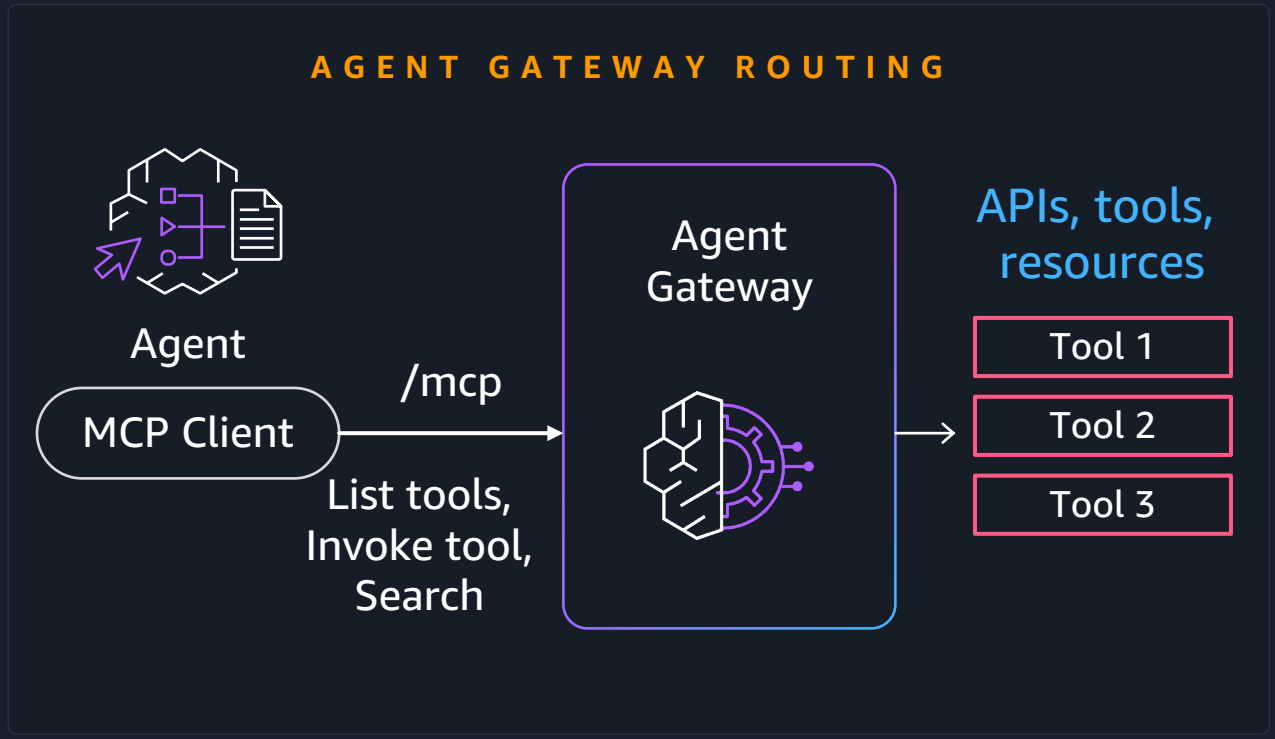
PILLAR 02 · TOOLS SYSTEM

工具系統——讓 AI 能存取外部的能力

Harness 的**第二層核心**——賦予 AI 與外部資訊互動、調用的能力

- **Tools**：包含調用 API、Web search 等等能力
- **MCPs**：存取內外部資料源
- **Skills**：動態載入正確知識

提供 LLM 執行所需的工具，讓它擁有足夠的知識與能力



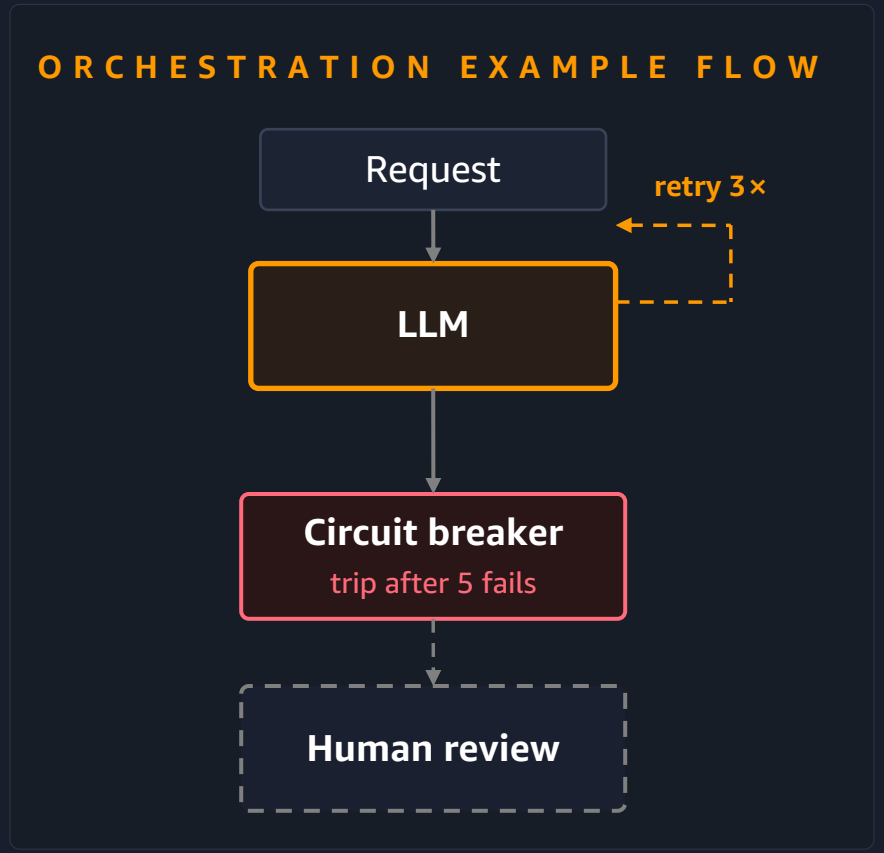
PILLAR 03 · ORCHESTRATION

執行編排——規劃 AI 每一步該做什麼

Harness 的**第三層核心**——編排 AI 的任務串接流程，決定工作的先後順序與分支

- **Flow control**：任務流程順序與分支條件
- **Retry**：錯誤重試與中斷條件
- **Human-in-the-loop**：高風險動作需要人類核准

編排 Agent 的執行流程，讓它擁有穩定可控的行為路徑



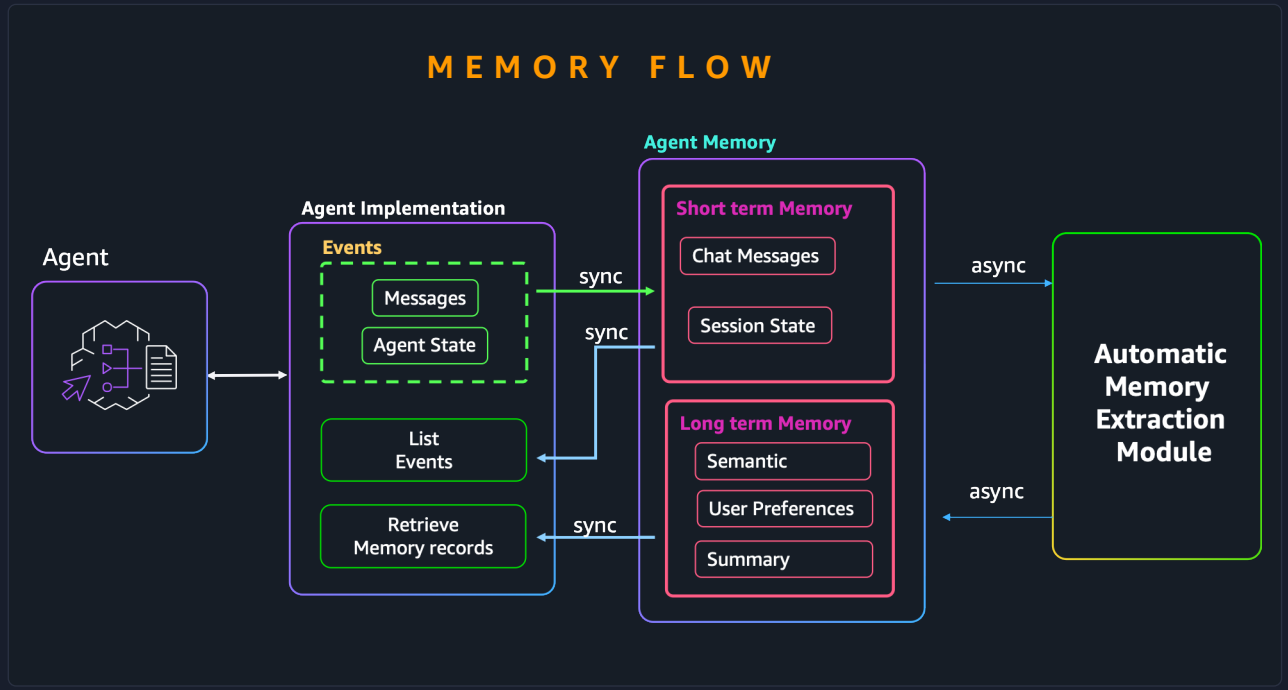
PILLAR 04 · MEMORY & STATUS

狀態與記憶——讓 AI 記得發生過什麼

Harness 的**第四層核心**——賦予 AI 能延續過往對話與任務，並回復進度的能力

- **Short-term memory**：紀錄目前任務的資訊與狀態
- **Long-term memory** ：紀錄跨任務的資訊與狀態
- **Execution Status**：紀錄任務執行的進度與資料

紀錄 Agent 任務狀態與記憶，讓它擁有延續脈絡與累積經驗的能力



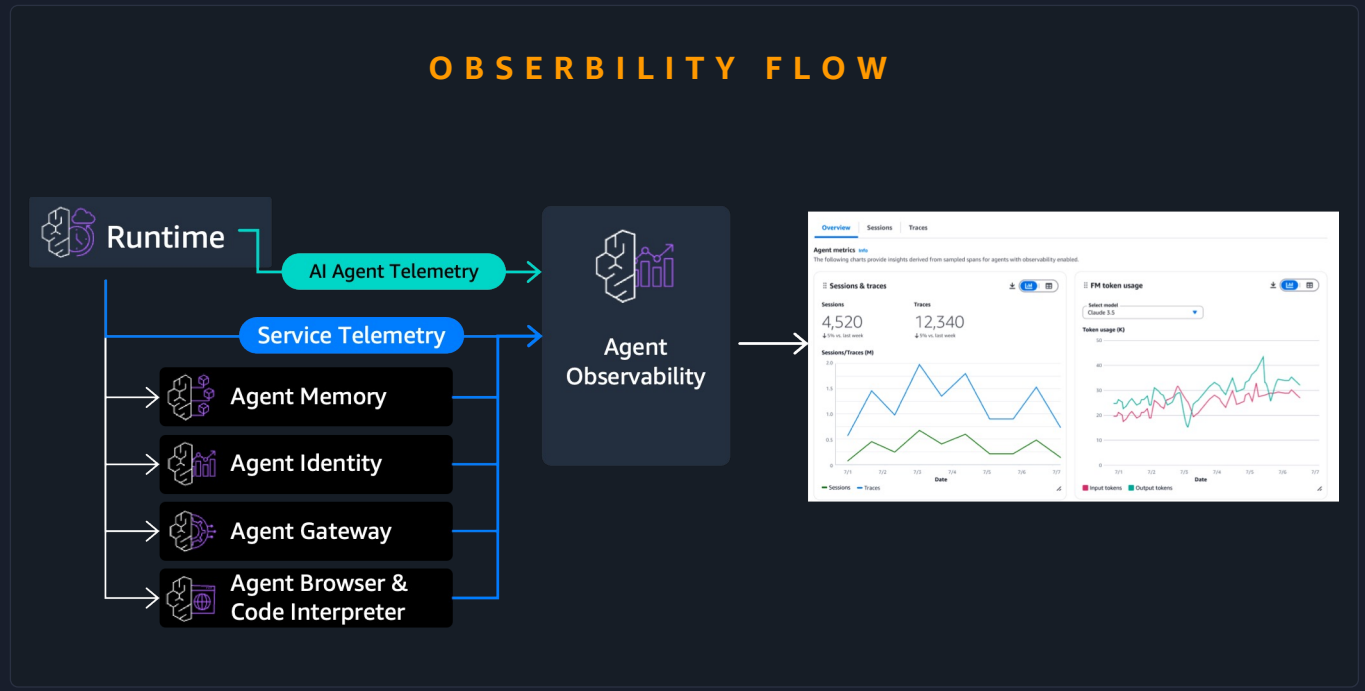
PILLAR 05 · OBSERVABILITY

可觀測性——讓 AI 任務可追蹤除錯與監控

Harness 的**第五層核心**——賦予 AI 在任務執行被追蹤、診斷、修正的能力

- **Sessions**：任務的完整上下文脈絡
- **Traces**：單次 request-response 的執行路徑
- **Spans**：每一個 operation 的起迄與狀態
- **Metrics**：紀錄任務執行的監控指標
- **Logs**：紀錄任務執行的日誌

記錄 Agent 每一步的行為與狀態，讓它擁有可被追蹤、診斷、除錯的透明度



PILLAR 6 · VERIFY & RECOVER

驗證與恢復——確保 AI 產出可信任、失敗可恢復

Harness 的**第六層核心**——賦予 AI 上線前被評測、出錯後被恢復的能力

- **Verification**：以單元、整合測試驗證內容的正確性
- **Evaluation**：以 LLM-as-a-Judge 評分 agent 的品質
- **Recovery**：自動重試、回滾與跳脫的機制

確保 LLM 的正確性、品質與韌性，保障生產環境的品質



THE ROADMAP

Harness — AI Engineering Roadmap

THE JOURNEY · 三段旅程

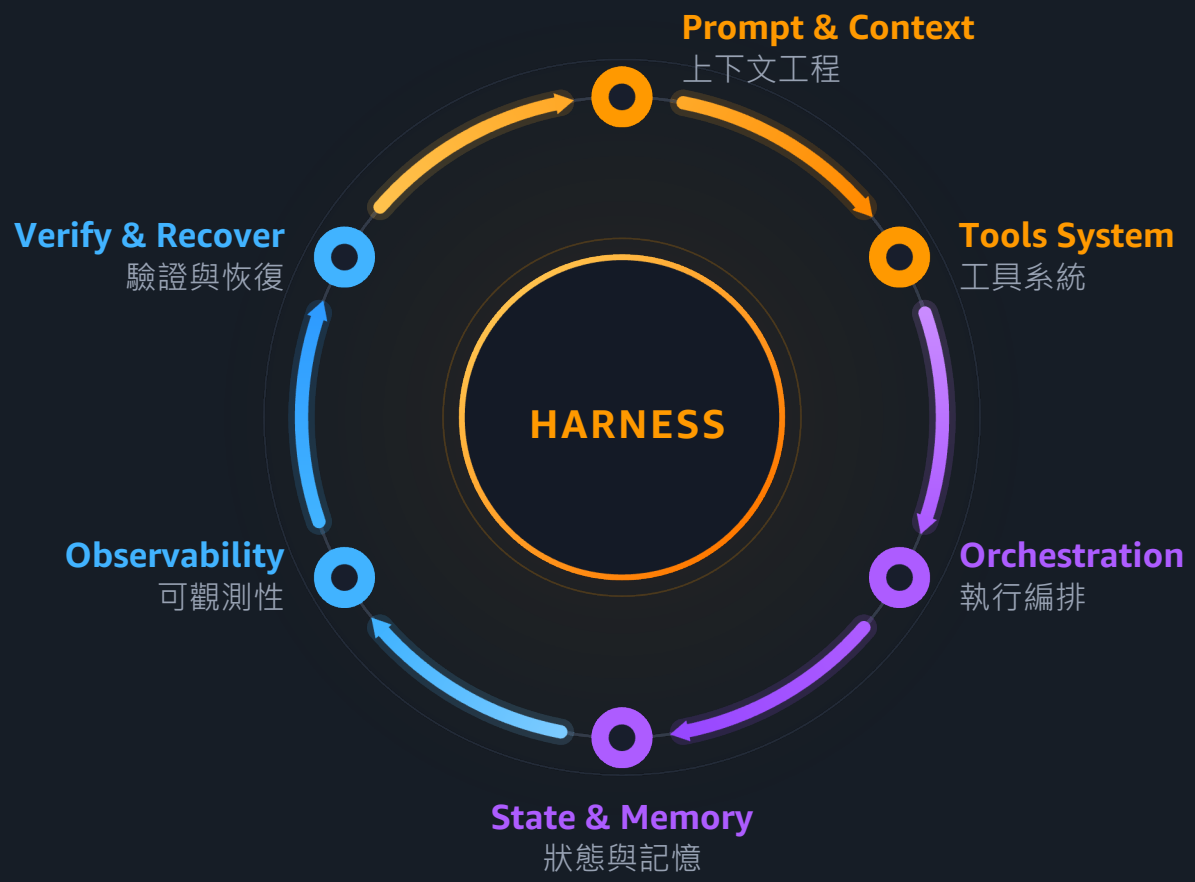
01 Prompt Engineering
學會「怎麼問」

↓

02 Context Engineering
對的資訊，對的時間點

↓

03 Harness Engineering
建立控制系統



BUILD & RUN · 實作路徑

開發 Harness Agent

↓

安全、可擴展地部署與運行

Planner → Generator → Evaluator

Prompt決定AI怎麼說

Context決定AI看到什麼

Harness決定AI能做什麼、做錯了怎麼辦

Thank you!



https://pulse.amazon/survey/AKJWGPNM